`IConfiguration`. This method grabs a chunk of the configuration and resets the namespace. Another way to get the latitude key is

```
var lat = builder.Configuration
    .GetSection("MapSettings")["DefaultLocation:Latitude"];
```

Accessing setting values this way is useful in Program.cs when you're defining your application. When you're setting up your application to connect to a database, for example, you'll often load a connection string from the `IConfiguration` object. You'll see a concrete example in chapter 12, which looks at Entity Framework Core.

If you need to access the configuration object in places other than Program.cs, you can use DI to inject it as a dependency into your service's constructor. But accessing configuration by using `string` keys this way isn't particularly convenient; you should try to use strongly typed configuration instead, as you'll see in section 10.3.

So far, this process probably feels a bit too convoluted and run-of-the-mill to load settings from a JSON file, and I'll grant you that it is. Where the ASP.NET Core configuration system shines is when you have multiple providers.

10.2.2 Using multiple providers to override configuration values

You've seen how to add a configuration provider to the `ConfigurationManager` and retrieve the configuration values, but so far, you've configured only a single provider. When you add providers, it's important to consider the order in which you add them, as that defines the order in which the configuration values will be added to the underlying dictionary. Configuration values from later providers overwrite values with the same key from earlier providers.